Atividade 02

- **Aluno:** Manuel Ferreira Junior
- **Disciplina:** Processamento Digital de Imagens
- **Obs.:** Questões com necessidade de calculo de convolução, estarão todas as expressões no link, para facilitar a visualização e leitura da atividade. Além disso, cada questão em anexo possui uma copia da sua operação que deve ser realizada. Cada sheet do link em anexo, possui no seu titulo a questão referente e operação referente.

Funções

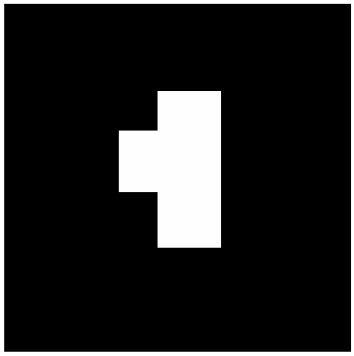
Abaixo, temos as funções criadas para utilizar ao decorrer da atividade, para as questões de implementação.

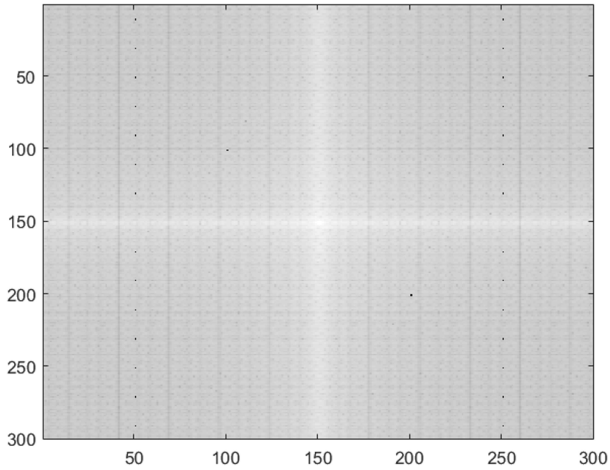
```
def find_contorno(imagem):
    """Verifica se a imagem contém a letra A maiúscula"""
    # Binariza a imagem (inversão: fundo branco, letras pretas)
    _, img_bin = cv2.threshold(imagem, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # Encontra contornos (possíveis letras)
    contornos, _ = cv2.findContours(img_bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    return contornos, img_bin

def tm_ccoef_normed(image: np.ndarray, template: np.ndarray) -> np.ndarray:
    """
    Calcula a correlação cruzada normalizada (TM_CCOEFF_NORMED) entre a imagem e o
template.

    Parâmetros:
    - image: imagem de entrada (2D, em escala de cinza)
    - template: template a ser comparado (2D, em escala de cinza)

    Retorna:
    - result: mapa de similaridade (valores entre -1 e 1)
    """
    img_h, img_w = image.shape
    tpl_h, tpl_w = template.shape

    # Média e T' do template
    template_mean = np.mean(template)
    template_prime = template - template_mean
    template_prime_squared_sum = np.sum(template_prime ** 2)

    # Saída terá tamanho reduzido
    result_h = img_h - tpl_h + 1
    result_w = img_w - tpl_w + 1
    result = np.zeros((result_h, result_w), dtype=np.float32)

    # Para cada posição possível (x, y) na imagem:
    for y in range(result_h):
        for x in range(result_w):
            region = image[y:(y + tpl_h), x:(x + tpl_w)]
            region_mean = np.mean(region)
            region_prime = region - region_mean
            region_prime_squared_sum = np.sum(region_prime ** 2)

            numerator = np.sum(template_prime * region_prime)
```

```
            denominator = np.sqrt(template_prime_squared_sum * region_prime_squared_sum)

            result[y, x] = numerator / denominator if denominator != 0 else 0.0

    return result
```

Questão 01

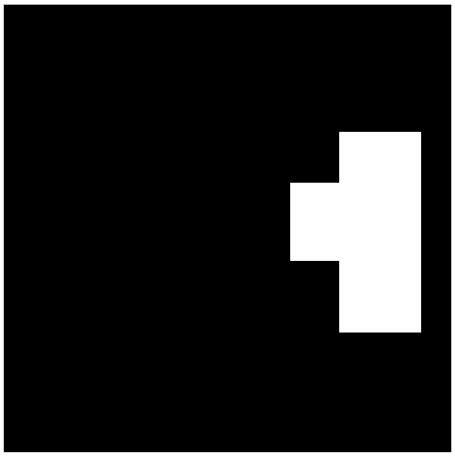
A imagem da esquerda tem a magnitude da sua transformada de Fourier apresentada à direita:





Como deve ser a mesma figura da magnitude da transformada de Fourier para as imagens abaixo? Considere que a única mudança, em relação à imagem apresentada acima e à esquerda, é o deslocamento do bloco branco. Justifique sua resposta.

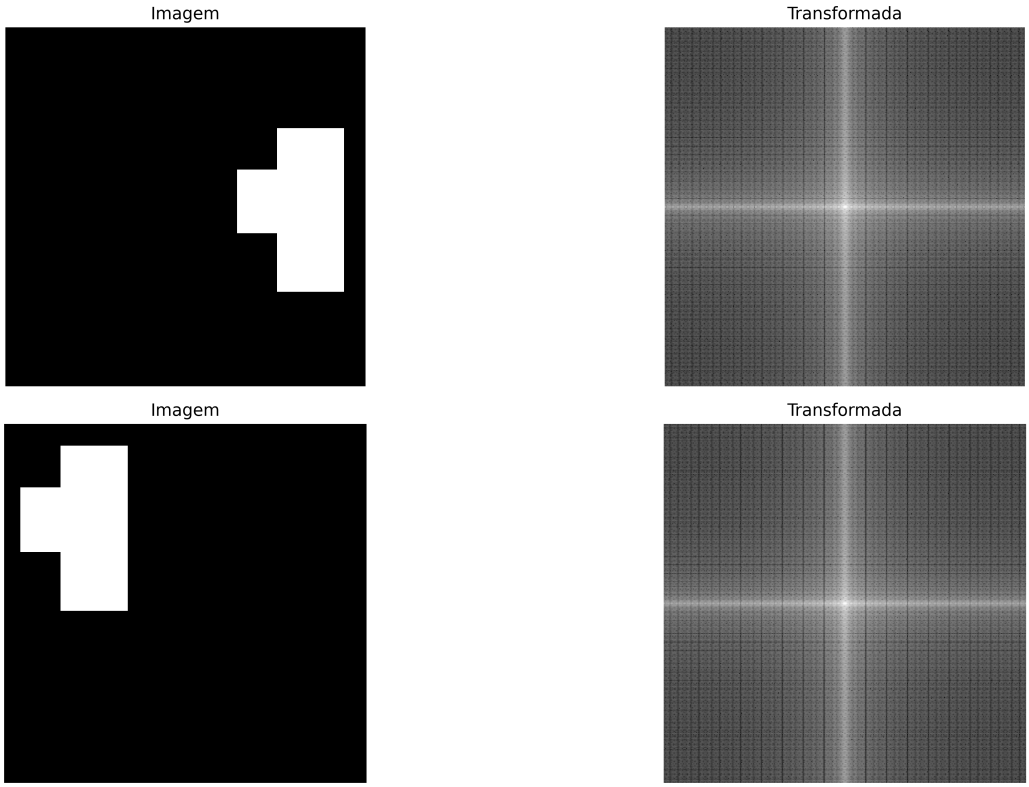
Obs: Você não tem a imagem original para calcular a transformada. Apenas especule sobre o resultado esperado.



R.:

A Transformada de Fourier tem como foco principal analisar a distribuicao das frequências da iamgem e não elementos dispostos no espaço da imagem, ou seja, a posição de qualquer conteúdo na imagem caso seja alterado, de forma igual a não alterar a frequências presente na imagem, espera-se que o resultado da transformada de fourier permaneça igual. Logo, a transformada de fourier independe da localização dos pixels sobre a imagem, caso não exista variação de intensidade entre as imagens.

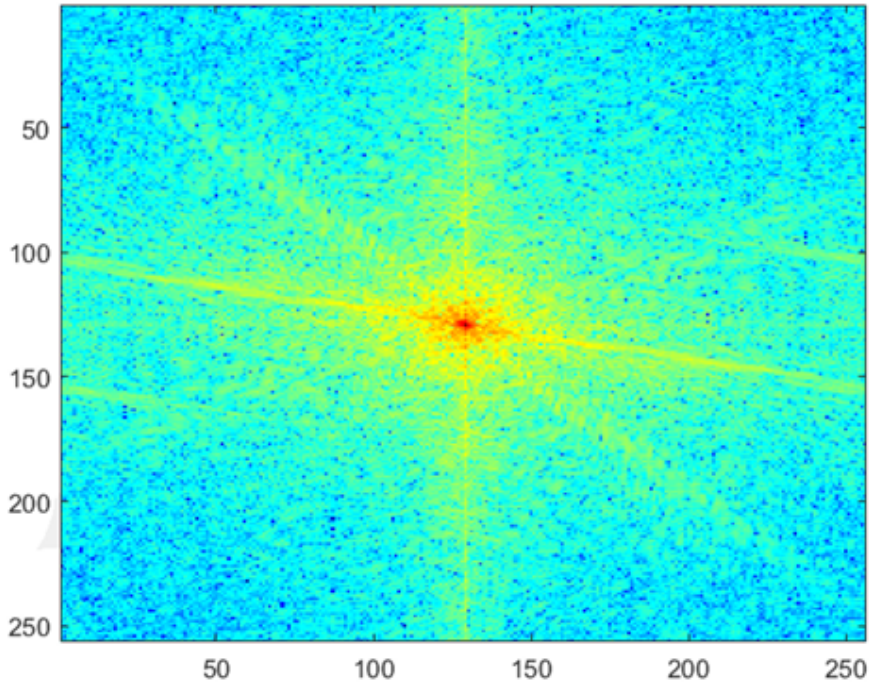
Na imagem abaixo, podemos ilustrar oque foi dito sobre o efeito da transformada de fourier:



Questão 02

Vimos, nos slides 16 e 17 da aula de filtragem, que o embaçamento de uma imagem provoca grandes mudanças na magnitude da transformada de Fourier dessa imagem. Considere a imagem abaixo:

Essa imagem gera a magnitude da transformada de Fourier a seguir:

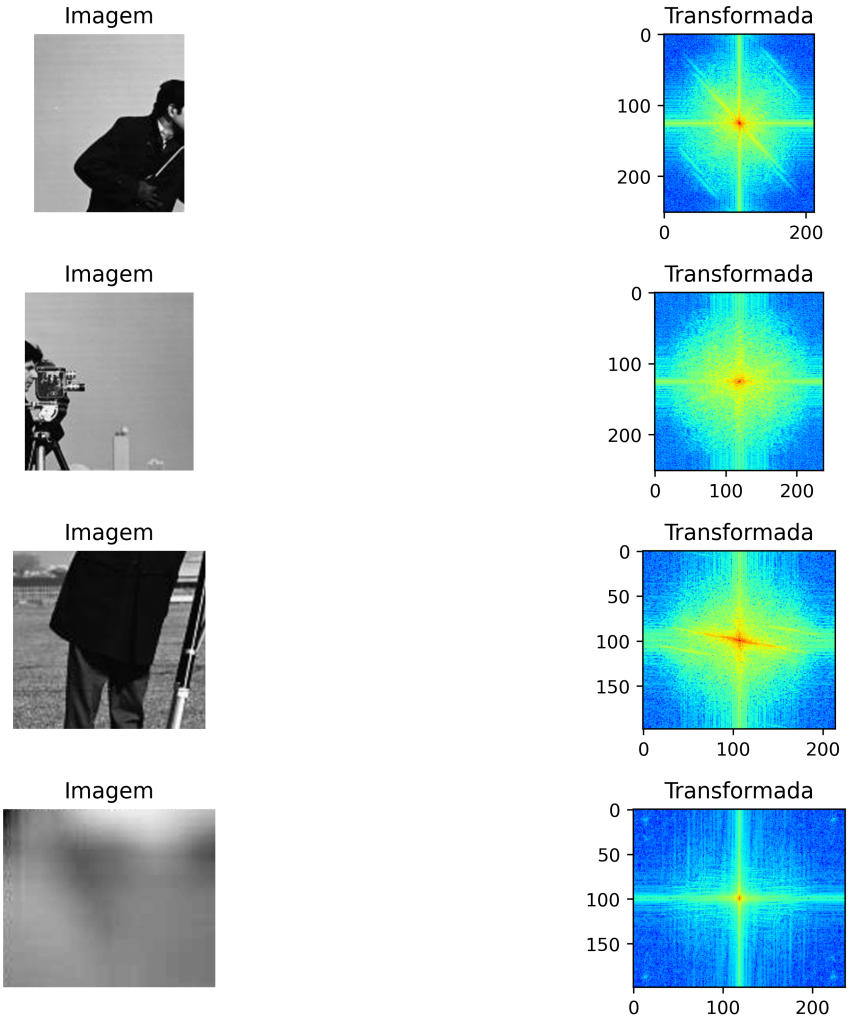


Em comparação com a imagem da direita do slide 17, da aula de filtragem, vemos que o borramento parcial no canto inferior da direita, apesar de bastante forte, não está tão nítido na transformada. Sugira uma estratégia para detectar que houve esse distúrbio em alguma parte (bem definida) da imagem.

R.:

Uma possivel estrategia para detectar esse tipo de anomalia localizada, como o borramento em regiões especificas, pode ser segmentar a imagem em regiões menores, em especial no exemplo da questão, em quatro quadrantes, ai então aplicar a transformada de Fourier em cada quadrante. A ideia é que, ao analisar o espectro individualmente para cada quadrante, será possível comparar as distribuições entre as regiões. O quadrante afetado pelo desfoque apresentará uma caracteristica diferente das outras, evidenciando o borramento.

Na imagem abaixo, da para notar a aplicação dessa solução de separação de quadrantes:



Questão 03

Considere o filtro passa baixa de Butterworth

$$H(u,v) = \frac{1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}$$

Calcule a expressão que representa o filtro passa alta de Butterworth, a partir desse filtro passa baixa. Dicas: Observe o que foi comentado no slide 57 da aula de filtragem. Lá, é para um filtro Gaussiano, mas o raciocínio é o mesmo. O resultado final a ser calculado está no slide 65; esse é o resultado a ser alcançado. Você deve calcular como chegar nele.

R.:

Sabendo que o filtro passa-alta nada mais é que o complementar do filtro passa baixa, temos então que:

$$H_{FPA}(u,v) = 1 - H_{FPB}(u,v)$$

Logo, substituindo a equação do filtro passa baixa de Butterworth, temos que:

$$H_{FPA}(u,v) = 1 - \frac{1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}} = \frac{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n} - 1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}} = \frac{\left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2 \cdot n}}$$

Então, sabendo que

$$\left[\frac{D(u,v)}{D_0}\right]^{2\cdot n} \cdot \left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n} = 1$$

logo:

$$H_{FPA}(u,v) = \frac{\left(\frac{D(u,v)}{D_0}\right)^{2\cdot n}}{\left(\frac{D(u,v)}{D_0}\right)^{2\cdot n}\left[1+\left(\frac{D(u,v)}{D_0}\right)^{-2\cdot n}\right]} = \frac{1}{1+\left(\frac{D(u,v)}{D_0}\right)^{-2\cdot n}}$$

Como

$$\left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n} = \left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}$$

Então temos:

$$H_{FPA}(u,v) = \frac{1}{1+\left[\frac{D(u,v)}{D_0}\right]^{-2\cdot n}} = \frac{1}{1+\left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}}$$

$$\therefore H_{FPA}(u,v) = \frac{1}{1+\left[\frac{D_0}{D(u,v)}\right]^{2\cdot n}} \square$$

Questão 04

A convolução discreta é uma operação comutativa. Ou seja:

f * h = h * f, onde * é a operação de convolução.

Comprove isso fazendo a convolução discreta dos dois filtros abaixo. Analise sua resposta.

(1 / 9) * f:

1 / 9	1 / 9	1 / 9
1 / 9	1 / 9	1 / 9
1 / 9	1 / 9	1 / 9

h:

-1	0	1
-1	0	1
-1	0	1

Ou seja, você deve calcular (e apresentar todos os cálculos) da convolução discreta das duas matrizes acima, operadas em ordens inversas: f * h e h * f.

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**.

1. **Para f * h:**

Primeiramente, vamos rotacionar a segunda matriz, sendo ela **h**, ficando:

h:

1	0	-1
1	0	-1
1	0	-1

Então, aplicando a convolução teremos:

1x1

1	0	-1		
1	0	-1		
1	0	-1 * 1 / 9	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9

1x2

1	0	-1		
1	0	-1		
1	0 * 1 / 9	-1 * 1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	

1x3

1	0	-1		
1	0	-1		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		

1x4

	1	0	-1	
	1	0	-1	
1 / 9	1 * 1 / 9	0 * 1 / 9	-1	
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		

1x5

		1	0	-1
		1	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		

CV¹ =

1x1	1x2	1x3	1x4	1x5
2x1	2x2	2x3	2x4	2x5
3x1	3x2	3x3	3x4	3x5
4x1	4x2	4x3	4x4	4x5
5x1	5x2	5x3	5x4	5x5

2x1

1	0	-1		
1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1 * 1 / 9	1 / 9	1 / 9
	1 / 9	1 / 9	1 / 9	

2x2

1	0	-1		
1	0 * (1 / 9)	-1 * 1 / 9	1 / 9	
1	0 * (1 / 9)	-1 * 1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	

2x3

1	0	-1		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 / 9	1 / 9	1 / 9		

2x4

	1	0	-1	
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
1 / 9	1 / 9	1 / 9		

2x5

		1	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 / 9		

CV¹ =

-0.11	-0.11	0	0.11	0.11
-0.22	-0.22	0	0.22	0.22
-0.33	-0.33	0	0.33	0.33
-0.22	-0.22	0	0.22	0.22
-0.11	-0.11	0	0.11	0.11

3x1

1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1 * 1 / 9	1 / 9	1 / 9

3x2

	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9
	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9
	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9

3x3

1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		

3x4

1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	

3x5

1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1

4x1

	1 / 9	1 / 9	1 / 9	
1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1		

4x2

		1 / 9	1 / 9	1 / 9
	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9
	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9
	1	0	-1	

4x3

1 / 9	1 / 9	1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1	0	-1		

4x4

1 / 9	1 / 9	1 / 9		
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
	1	0	-1	

4x5

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 * 1 / 9	0	-1
1 / 9	1 / 9	1 * 1 / 9	0	-1
		1	0	-1

5x1

	1 / 9	1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	
1	0	-1 * 1 / 9	1 / 9	1 / 9
1	0	-1		
1	0	-1		

5x2

		1 / 9	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9
	1	0 * (1 / 9)	-1 * 1 / 9	1 / 9
	1	0	-1	
	1	0	-1	

5x3

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
1 * 1 / 9	0 * (1 / 9)	-1 * 1 / 9		
1	0	-1		
1	0	-1		

5x4

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
1 / 9	1 * 1 / 9	0 * (1 / 9)	-1	
	1	0	-1	
	1	0	-1	

5x5

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 * 1 / 9	0	-1
		1	0	-1
		1	0	-1

Logo, a matriz resultante da convolução será:

$$f * h = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix}$$

2. **Para h * f:**

Primeiramente, vamos rotacionar a segunda matriz, sendo ela **f**, devido a sua natureza, a rotação dela é igual a original, logo aplicando a revolução teremos:

1x1

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	-1 * 1 / 9	0	1
		-1	0	1
		-1	0	1

1x2

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
1 / 9	-1 * 1 / 9	0 * (1 / 9)	1	
	-1	0	1	
	-1	0	1	

1x3

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		
-1 * 1 / 9	0 * (1 / 9)	1 * 1 / 9		
-1	0	1		
-1	0	1		

1x4

	1 / 9	1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	
-1	0 * 1 / 9	1 * 1 / 9	1 / 9	
-1	0	1		
-1	0	1		

1x5

		1 / 9	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9
-1	0	1 * 1 / 9	1 / 9	1 / 9
-1	0	1		
-1	0	1		

CV² =

1x1	1x2	1x3	1x4	1x5
2x1	2x2	2x3	2x4	2x5
3x1	3x2	3x3	3x4	3x5
4x1	4x2	4x3	4x4	4x5
5x1	5x2	5x3	5x4	5x5

2x1

1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	-1 * 1 / 9	0	1
1 / 9	1 / 9	-1 * 1 / 9	0	1
		-1	0	1

2x2

1 / 9	1 / 9	1 / 9		
1 / 9	-1 * 1 / 9	0 * (1 / 9)	1	
1 / 9	-1 * 1 / 9	0 * (1 / 9)	1	
	-1	0	1	

2x3

1 / 9	1 / 9	1 / 9		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
-1	0	1		

2x4

	1 / 9	1 / 9	1 / 9	
-1	0 * 1 / 9	1 * (1 / 9)	1 / 9	
-1	0 * 1 / 9	1 * (1 / 9)	1 / 9	
-1	0	1		

2x5

		1 / 9	1 / 9	1 / 9
-1	0	1 * (1 / 9)	1 / 9	1 / 9
-1	0	1 * (1 / 9)	1 / 9	1 / 9
-1	0	1		

CV² =

-0.11	-0.11	0	0.11	0.11
-0.22	-0.22	0	0.22	0.22
-0.33	-0.33	0	0.33	0.33
-0.22	-0.22	0	0.22	0.22
-0.11	-0.11	0	0.11	0.11

3x1

1 / 9	1 / 9	-1 * 1 / 9	0	1
1 / 9	1 / 9	-1 * 1 / 9	0	1
1 / 9	1 / 9	-1 * 1 / 9	0	1

3x2

	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1
	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1
	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1

3x3

-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		

3x4

-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	
-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	
-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	

3x5

-1	0	1 * (1 / 9)	1 / 9	1 / 9
-1	0	1 * (1 / 9)	1 / 9	1 / 9
-1	0	1 * (1 / 9)	1 / 9	1 / 9

4x1

		-1	0	1
1 / 9	1 / 9	-1 * (1 / 9)	0	1
1 / 9	1 / 9	-1 * (1 / 9)	0	1
1 / 9	1 / 9	1 / 9		

4x2

		-1	0	1
	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1
	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1
	1 / 9	1 / 9	1 / 9	

4x3

-1	0	1		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
1 / 9	1 / 9	1 / 9		

4x4

-1	0	1		
-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	
-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	
	1 / 9	1 / 9	1 / 9	

4x5

-1	0	1		
-1	0	1 * (1 / 9)	1 / 9	1 / 9
-1	0	1 * (1 / 9)	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9

5x1

		-1	0	1
		-1	0	1
1 / 9	1 / 9	-1 * (1 / 9)	0	1
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		

5x2

		-1	0	1
		-1	0	1
	1 / 9	-1 * (1 / 9)	0 * (1 / 9)	1
	1 / 9	1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	

5x3

-1	0	1		
-1	0	1		
-1 * (1 / 9)	0 * (1 / 9)	1 * (1 / 9)		
1 / 9	1 / 9	1 / 9		
1 / 9	1 / 9	1 / 9		

5x4

-1	0	1		
-1	0	1		
-1	0 * (1 / 9)	1 * (1 / 9)	1 / 9	
	1 / 9	1 / 9	1 / 9	
	1 / 9	1 / 9	1 / 9	

5x5

-1	0	1		
-1	0	1		
-1	0	1 * (1 / 9)	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9
		1 / 9	1 / 9	1 / 9

Analogo ao item anterior, temos:

$$h * f = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix}$$

3. **Conclusão:**

Logo, temos de fato que f * h = h * f.

$$f * h = h * f = \begin{bmatrix} -0.11 & -0.11 & 0 & 0.11 & 0.11 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.33 & -0.33 & 0 & 0.33 & 0.33 \\ -0.22 & -0.22 & 0 & 0.22 & 0.22 \\ -0.11 & -0.11 & 0 & 0.11 & 0.11 \end{bmatrix} \square$$

Questão 05

Calcule uma janela para um possível filtro Gaussiano através da convolução discreta de três filtros Box, como abaixo:

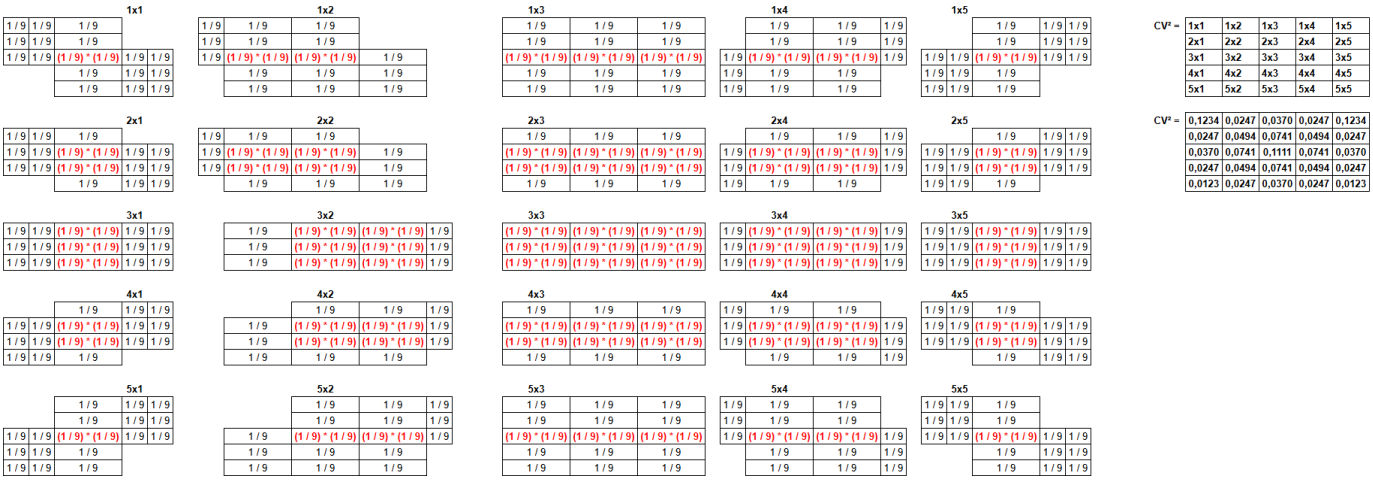
$$\text{Filtro Box} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**. Vamos tomar como **B = Filtro Box**, para simplificar.

1) B * B

Calculando, devido a simetria de B, a rotação será igual, logo temos:



Resultando em:

$$B * B = \begin{bmatrix} 0.0123 & 0.0247 & 0.0370 & 0.0247 & 0.0123 \\ 0.0247 & 0.0494 & 0.0741 & 0.0494 & 0.0247 \\ 0.0370 & 0.0741 & 0.1111 & 0.0741 & 0.0370 \\ 0.0247 & 0.0494 & 0.0741 & 0.0494 & 0.0247 \\ 0.0123 & 0.0247 & 0.0370 & 0.0247 & 0.0123 \end{bmatrix}$$

2) (B * B) * B

Analogo ao item anterior, devido a simetria de B, teremos rotação igual, logo temos:

Resultado em:

$$(B * B) * B = \begin{bmatrix} 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0096 & 0.0288 & 0.0576 & 0.0672 & 0.0576 & 0.0288 & 0.0096 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \end{bmatrix}$$

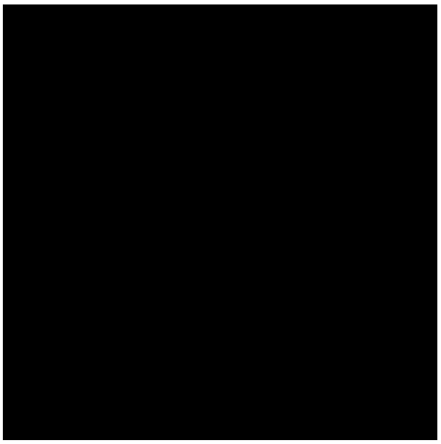
3. **Conclusão:**

Temos o produto resultante abaixo:

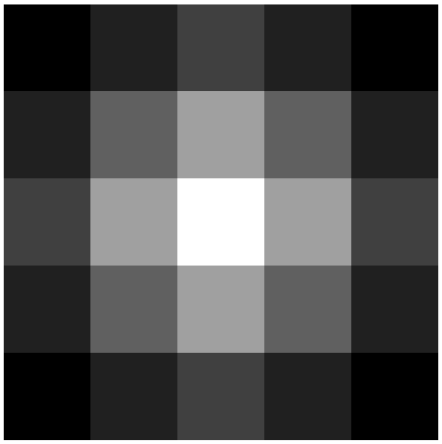
$$(B * B) * B = \begin{bmatrix} 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0096 & 0.0288 & 0.0576 & 0.0672 & 0.0576 & 0.0288 & 0.0096 \\ 0.0082 & 0.0247 & 0.0494 & 0.0576 & 0.0494 & 0.0247 & 0.0082 \\ 0.0041 & 0.0123 & 0.0247 & 0.0288 & 0.0247 & 0.0123 & 0.0041 \\ 0.0014 & 0.0041 & 0.0082 & 0.0096 & 0.0082 & 0.0041 & 0.0014 \end{bmatrix}$$

Ilustrando graficamente o efeito desse filtro box, temos:

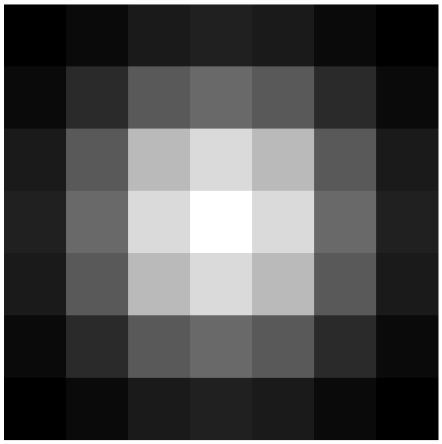
Matriz Original



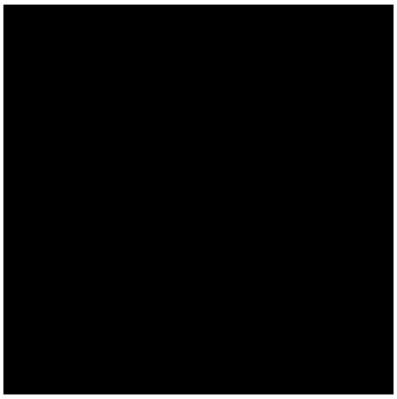
B * B



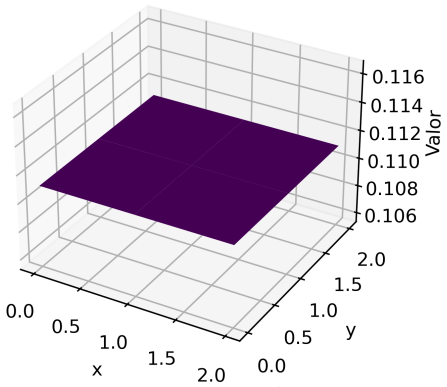
(B * B) * B



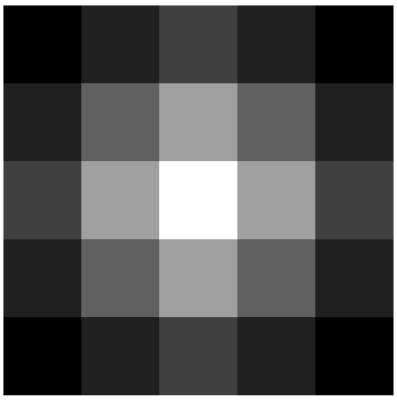
Filtro B - Imagem



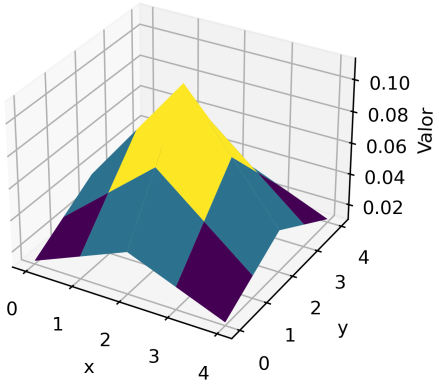
Filtro B - Superfície



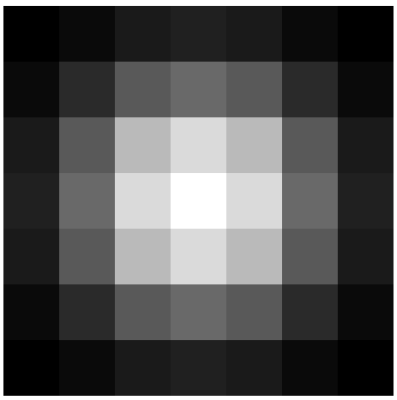
Filtro B * B - Imagem



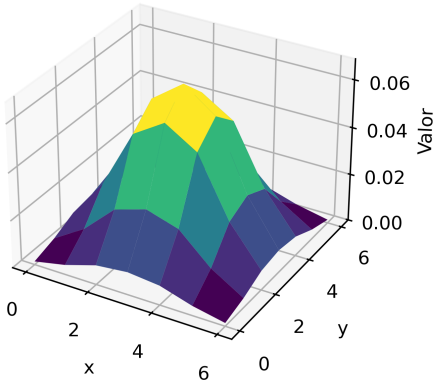
Filtro B * B - Superfície



Filtro (B * B) * B - Imagem



Filtro (B * B) * B - Superfície



Questão 06

Considere a imagem abaixo, uma imagem em 16 tons de cinza. Apresente o resultado da aplicação de um filtro Box 3x3 (o mesmo Box da questão anterior) nessa imagem. Apresente os cálculos e explique todas as decisões tomadas para realizar a filtragem, observando que sua saída deve também ser uma imagem em 16 tons de cinza.

1	3	0
2	2	3
1	3	1

R.:

Obs.: Durante a questão, será marcado em **vermelho** os valores a serem somados para compor a sua localização na matriz **m x n**.

Como visto nas questões anteriores, a rotação da matriz de filtro box 3x3 é simétrica, então sua rotação será igual a ela mesma.

Como visto na aula de filtragem, apartir do slide 98, vamos aplicar uma convolução entre um filtro e uma imagem, logo usaremos a **Correlação Cruzada**, definindo como estratégia para a borda será a extensão nula, e apos aplicar o processo, multiplicaremos a constante 1 / 9.

Logo:

1x1

1	1	1	
1	1 * 1	1 * 3	0
1	1 * 2	1 * 2	3
	1	3	1

1x2

1	1	1
1 * 1	1 * 3	1 * 0
1 * 2	1 * 2	1 * 3
1	3	1

1x3

	1	1	1
1	1 * 3	1 * 0	1
2	1 * 2	1 * 3	1
1	3	1	

CV¹ =

1x1	1x2	1x3
2x1	2x2	2x3
3x1	3x2	3x3

CV¹ =

8	11	8
12	16	12
8	12	9

2x1

1	1 * 1	1 * 3	0
1	1 * 2	1 * 2	3
1	1 * 1	1 * 3	1

2x2

1 * 1	1 * 3	1 * 0
1 * 2	1 * 2	1 * 3
1 * 1	1 * 3	1 * 1

2x3

1	1 * 3	1 * 0	1
2	1 * 2	1 * 3	1
1	1 * 3	1 * 1	1

3x1

	1	3	0
1	1 * 2	1 * 2	3
1	1 * 1	1 * 3	1
1	1	1	

3x2

1	3	0
1 * 2	1 * 2	1 * 3
1 * 1	1 * 3	1 * 1
1	1	1

3x3

1	3	0	
2	1 * 2	1 * 3	1
1	1 * 3	1 * 1	1
	1	1	1

Então temos:

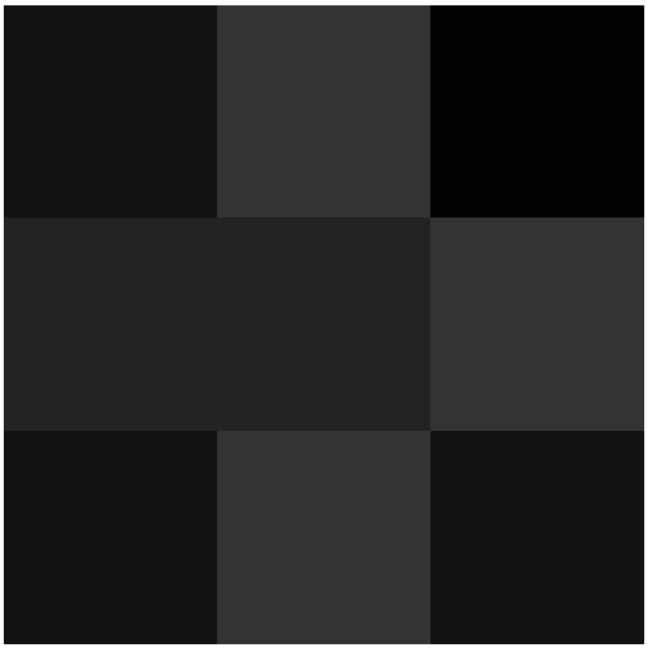
$$\frac{1}{9} \cdot (\text{Img} * \text{B}) = \frac{1}{9} \cdot \begin{bmatrix} 8 & 11 & 8 \\ 12 & 16 & 12 \\ 8 & 12 & 9 \end{bmatrix} = \begin{bmatrix} 0.8889 & 1.2222 & 0.8889 \\ 1.3333 & 1.7778 & 1.3333 \\ 0.8889 & 1.3333 & 1 \end{bmatrix}$$

Aplicando os devidos arredondamentos, temos:

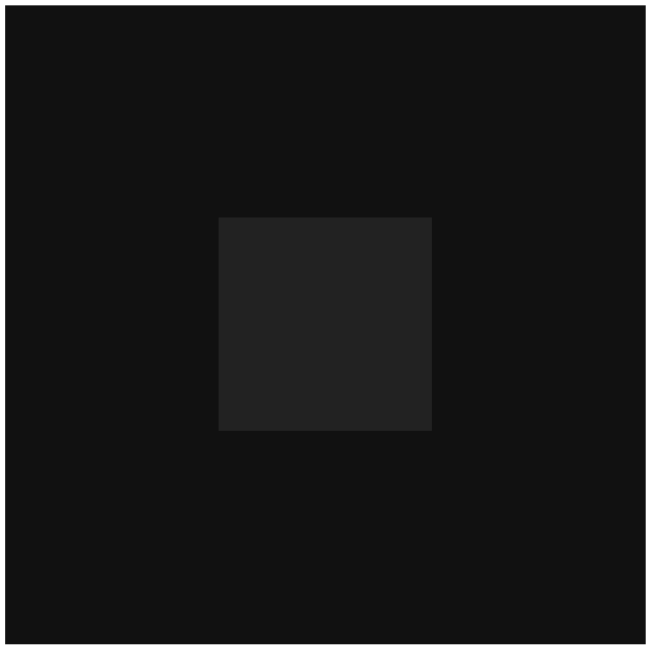
$$\frac{1}{9} \cdot (\text{Img} * \text{B}) = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

A ilustração gráfica do procedimento realizado esta abaixo:

Imagem Original



Com Filtro Box 3x3



Questão 07

Disserte sobre a seguinte afirmação:

"As operações morfológicas de Erosão e Dilatação, aplicadas com um mesmo elemento estruturante, não são, necessariamente, operações inversas uma da outra."

R.:

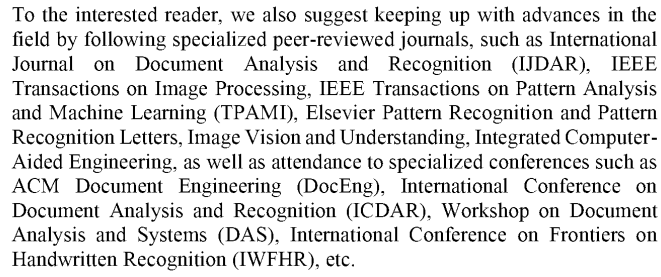
Enquanto a dilatação tem como objetivo expandir um objeto, a erosão visa torná-lo mais estreito. Essas operações não são necessariamente inversas, e isso pode ser exemplificado por um caso em que a erosão elimina um ponto isolado. Se uma dilatação for aplicada em seguida, esse ponto não poderá ser recuperado, uma vez que a dilatação expande apenas regiões já existentes. Se o ponto era isolado e não está mais presente na imagem, não há nada a ser expandido.

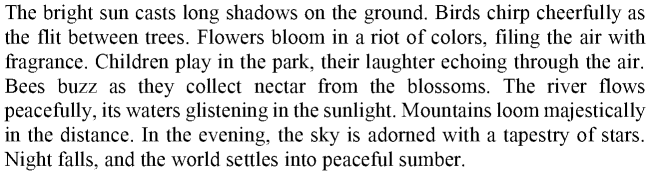
O cenário oposto também pode ocorrer: ao se aplicar uma dilatação, duas regiões próximas podem se unir. Após isso, uma erosão não será capaz de separá-las novamente, pois agora a imagem possui apenas uma única região contínua. Assim, não é possível aplicar uma suavização que recupere a separação original entre essas regiões.

Ao inves de serem tratadas como operações inversas, elas são complementares, os quais são definidos como Abertura (Suavisação de contornos em objetos, Remoção de ramos em objetos e Expansão de regiões de preto) e Fechamento (Preenchimento de falhas em regiões com contorno, diminuição de áreas de preto), a qual a primeira é a aplicação de uma erosão seguida de uma dilatação e a outra é uma dilatação seguida de uma erosão, ambas com o uso de um mesmo elemento estruturante.

Questão 08

Considere as imagens Book_1.png e Book_2.png disponibilizadas. Utilizando apenas técnicas de processamento de imagens, crie um algoritmo que verifique se essas imagens possuem a letra A ou não. Apenas o A maiúsculo deve ser procurado e não precisa retornar quantos têm; apenas se tem ou não. Observe que as imagens estão em preto e branco.


To the interested reader, we also suggest keeping up with advances in the field by following specialized peer-reviewed journals, such as International Journal on Document Analysis and Recognition (IJDAR), IEEE Transactions on Image Processing, IEEE Transactions on Pattern Analysis and Machine Learning (TPAMI), Elsevier Pattern Recognition and Pattern Recognition Letters, Image Vision and Understanding, Integrated Computer-Aided Engineering, as well as attendance to specialized conferences such as ACM Document Engineering (DocEng), International Conference on Document Analysis and Recognition (ICDAR), Workshop on Document Analysis and Systems (DAS), International Conference on Frontiers on Handwritten Recognition (IWFHR), etc.

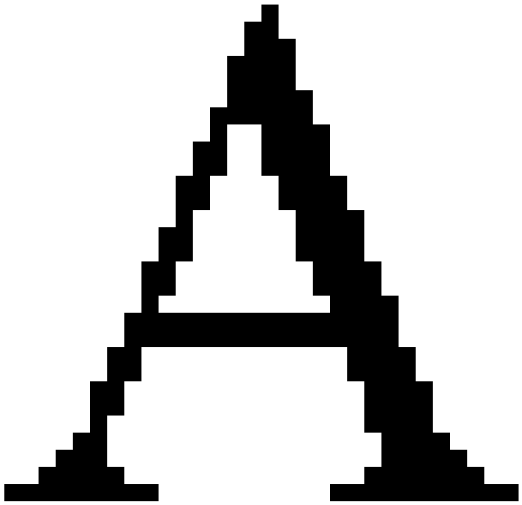

The bright sun casts long shadows on the ground. Birds chirp cheerfully as the flit between trees. Flowers bloom in a riot of colors, filing the air with fragrance. Children play in the park, their laughter echoing through the air. Bees buzz as they collect nectar from the blossoms. The river flows peacefully, its waters glistening in the sunlight. Mountains loom majestically in the distance. In the evening, the sky is adorned with a tapestry of stars. Night falls, and the world settles into peaceful sumber.

R.:

Obs.: O notebook criado para esse item pode ser encontrado neste link. Nele pode ser encontrado a resolução em um formato de notebook, se for do interesse.

Antes de prosseguir a atividade, é necessário a definição de alguns pontos, sendo eles os abaixos:

1. **Definição de um target ou template:** para podermos comparar contornos e objetos encontrados pelo algoritmo em uma imagem, se faz necessário um objeto para comparação, ou seja, um template do objeto de interesse, que nesse caso será a letra **A**. O template utilizado pode ser visualizado abaixo:



2. **Métrica para definir similaridade:** durante a questão, se faz necessário um meio de mensurar a similaridade entre uma imagem e o template, para tanto será definido a métrica `TM_CCOEFF_NORMED`, disponibilizada pelo proprio *OpenCV*, a qual é dada pela expressão abaixo:

$$R(x,y) = \frac{\sum_{x',y'}(T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$$

Sendo:

$$\begin{cases} x, y, & \text{coordenadas relativas do ponto na imagem;} \\ x', y' & \text{coordenadas relativas do ponto no template;} \\ T'(x',y'), & \text{Valor do pixel (com T centralizado na média) no template para o ponto } (x',y'); \\ I'(x+x',y+y') & \text{Representando o pixel na imagem de entrada (com I centralizado) em uma posição deslocada.} \end{cases}$$

3. **Definição de corte para a similaridade:** será necessário a seleção de um ponto de corte da similaridade para definir um objeto contornado como uma letra **A**, para tanto será selecionado um ponto de corte de 0.5, ou seja:

$$\text{Resultado} = \begin{cases} \text{Letra 'A' encontrada,} & \text{se } thres \geq 0.5 \\ \text{Letra 'A' não encontrada,} & \text{se } thres < 0.5 \end{cases}$$

1) 1º Imagem

1.1) Leitura das imagens

Primeiro, vamos realizar a leitura das imagens seguindo o código abaixo:

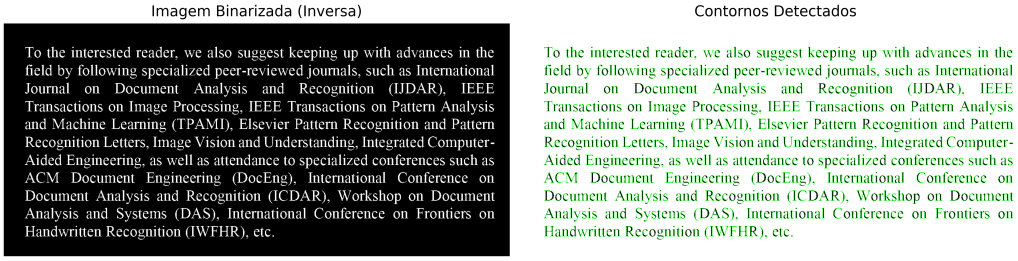
```
# Leitura de imagens
file_path_template_A = str(path_assets / "atv02-q08-01_template_A.png")
image_path = str(path_assets / "atv02_lista02-assets" / "Book_1.png")

image_book = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
image_template = cv2.imread(file_path_template_A, cv2.IMREAD_GRAYSCALE)

# parametros para salvar
contents = {
    "contornos": [],
    "letra": [],
    "letra_norm": [],
```

```
    "similarity": [],
    "check_is_valid": []
}
```

1.2) **Ilustração do processo de binarização e contornos**

Abaixo temos a imagem binarizada, apos a inversão e ao lado o contorno de cada letra encontrada.

```
# Mostra imagem binarizada e com contornos lado a lado
fig1, ax1 = plt.subplots(1, 2, figsize=(12, 5))
ax1[0].set_title("Imagem Binarizada (Inversa)")
ax1[0].imshow(img_bin, cmap='gray')
ax1[0].axis('off')

ax1[1].set_title("Contornos Detectados")
ax1[1].imshow(imagem_com_contornos, cmap='gray')
ax1[1].axis('off')

fig1.tight_layout()
fig1.savefig(path_assets / "atv02-q08-i01_k03.png", dpi=400, bbox_inches='tight')
plt.show()
```



1.3) **Busca de similaridade com o template utilizado**

Abaixo, será aplicado a cada contorno encontrado um calculo de similaridade mencionado anteriormente (`TM_CCOEFF_NORMED`), com a implementação disponibilizada pela *OpenCV*.

```
fig, ax = plt.subplots(10, 10, figsize=(18, 16))
ax = ax.flatten()
for i, contorno in enumerate(contornos):
    x, y, w, h = cv2.boundingRect(contorno)

    letra = img_bin[y:(y + h), x:(x + w)]  # recorta a letra da iamgem
    letra_resized = cv2.resize(
        letra,
        (image_template.shape[1], image_template.shape[0])
    )  # redimensiona para o tamanho do template

    letra_norm = 1 - letra_resized / 255.0 # re normalizando para retornar com o fundo
branco e letra preta

    # Outro formato de uso usando a biblioteca OpenCV
################################################################
    # res = cv2.matchTemplate(
    #     letra_norm.astype(np.float32),
    #     image_template.astype(np.float32),
    #     cv2.TM_CCOEFF_NORMED
    # )[0][0]  # calculo de similaridade explicitado
```

```
##########################################################################################
#########################
    res = tm_ccoef_normed(letra_norm, image_template)  # calculo de similaridade
    # simililarity = res[0][0]
    simililarity = res[0][0]

    contents["contornos"].append(contorno)
    contents["letra"].append(letra)
    contents["letra_norm"].append(letra_norm)
    contents["similarity"].append(simililarity)

    if w < 10 or h < 10:  # contornos muito pequenos não serão mostrados
        if i < (10 * 10):
            ax[i].axis("off")
        contents["check_is_valid"].append(False)
        continue
    contents["check_is_valid"].append(True)

    if i < (10 * 10):
        ax[i].imshow(letra_norm, cmap='gray')
        ax[i].axis("off")
        ax[i].set_title(f"Sim: {simililarity:.2f}")
fig.savefig(path_assets / "atv02-q08-i01_k04.png", dpi=400, bbox_inches='tight')
fig.suptitle("Amostra de letras e similaridade calculada")
```

1.4) **Threshold para similaridade**

Como dito anteriormente, vamos considerar um *threshold* de similaridade de 0.5, para definir que o objeto encontrado pelo contorno é a letra **A**. Podemos visualizar logo abaixo:

```
p = 0.5
ks = np.where(
  (np.array(contents["similarity"]) >= p) & (np.array(contents["check_is_valid"]) ==
True)
)[0]  # achando os indices das letras com similirade >= 0.5
n = math.ceil(math.sqrt(len(ks)))

if n != 0:
```

```
    fig, axes = plt.subplots(
        n,
        n,
        figsize=(n * 3, n * 3)
    )


    for value, _axes in zip(ks, axes.flatten()):
        _axes.imshow(contents["letra_norm"][value], cmap="gray")
        _axes.set_title(f"Contorno: {value}")

    for _axes in axes.flatten():
        _axes.axis('off')

    text = f"Letra 'A' {'encontrada' if len(ks) >= 1 else 'inexistente'}"
    text += f" | Total: {len(ks)}"
    fig.suptitle(text, fontsize=16, weight='bold')
    fig.savefig(path_assets / "atv02-q08-i01_k05.png", dpi=400, bbox_inches='tight')
    plt.show()
else:
    fig, axes = plt.subplots(
        1,
        1,
        figsize=(1, 8)
    )
    axes.set_title("Letra 'A' não encontrada")
    axes.axis('off')
    fig.savefig(path_assets / "atv02-q08-i01_k05.png", dpi=400, bbox_inches='tight')
```

1.5) **Conclusão**

Ao aplicarmos o algoritmo anterior, para a imagem do `Book_1.png`, foi visto que ele possuí a letra **A** em sua imagem. Um ponto a mais, com o formato da implementação utilizada, é possível listar o número de **A**'s encontrados, totalizando 10 para a imagem `Book_1.png`.

2) **2º Imagem**

O procedimento para a segunda imagem será analogo a primeira imagem, mudando apenas o objeto na leitura.

2.1) **Leitura das imagens**

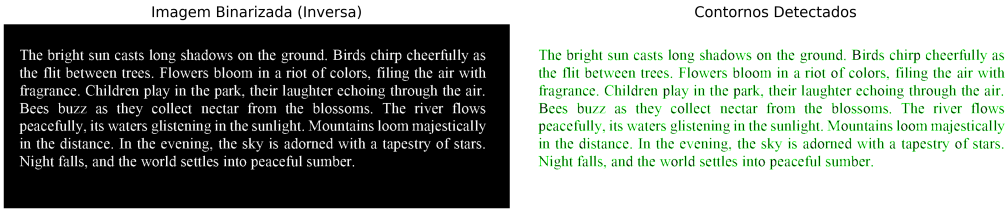
Primeiro, vamos realizar a leitura das imagens seguindo o código abaixo:

```
# Leitura de imagens
file_path_template_A = str(path_assets / "atv02-q08-01_template_A.png")
image_path = str(path_assets / "atv02_lista02-assets" / "Book_2.png")

image_book = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
image_template = cv2.imread(file_path_template_A, cv2.IMREAD_GRAYSCALE)

# parametros para salvar
contents = {
    "contornos": [],
    "letra": [],
    "letra_norm": [],
    "similarity": [],
    "check_is_valid": []
}
```

2.2) **Ilustração do processo de binarização e contornos**

Como no item anterior (1.2), fizemos a mesma inversão de cores, com a binarização, e ao lado o contorno de cada letra encontrada:

```
# Pega os contornos e a imagem binarizada
contornos, img_bin = find_contorno(image_book)

# Faz uma cópia colorida da imagem original (para desenhar colorido)
imagem_com_contornos = cv2.cvtColor(image_book, cv2.COLOR_GRAY2BGR)

# Desenha os contornos
_ = cv2.drawContours(imagem_com_contornos, contornos, -1, (0, 255, 0), 1)

# Mostra imagem binarizada e com contornos lado a lado
fig1, ax1 = plt.subplots(1, 2, figsize=(12, 5))
ax1[0].set_title("Imagem Binarizada (Inversa)")
ax1[0].imshow(img_bin, cmap='gray')
ax1[0].axis('off')

ax1[1].set_title("Contornos Detectados")
ax1[1].imshow(imagem_com_contornos, cmap='gray')
ax1[1].axis('off')

fig1.tight_layout()
fig1.savefig(path_assets / "atv02-q08-i02_k03.png", dpi=400, bbox_inches='tight')
plt.show()
```



2.3) Busca de similaridade com o template utilizado

Aplicando a mesma logica de similaridade do item (1.3), utilizando a similaridade escolhida. Podemos visualizar a amostra de alguns objetos encontrados abaixo:

```
fig, ax = plt.subplots(10, 10, figsize=(18, 16))
ax = ax.flatten()
for i, contorno in enumerate(contornos):
    x, y, w, h = cv2.boundingRect(contorno)

    letra = img_bin[y:(y + h), x:(x + w)]  # recorta a letra da iamgem
    letra_resized = cv2.resize(
        letra,
        (image_template.shape[1], image_template.shape[0])
    )  # redimensiona para o tamanho do template

    letra_norm = 1 - letra_resized / 255.0 # re normalizando para retornar com o fundo
branco e letra preta

    # Outro formato de uso usando a biblioteca OpenCV
################################################################
    # res = cv2.matchTemplate(
    #     letra_norm.astype(np.float32),
    #     image_template.astype(np.float32),
    #     cv2.TM_CCOEFF_NORMED
    # )[0][0]  # calculo de similaridade explicitado

#######################################################################################
#########################
    res = tm_ccoef_normed(letra_norm, image_template)  # calculo de similaridade
    simililarity = res[0][0]

    contents["contornos"].append(contorno)
    contents["letra"].append(letra)
    contents["letra_norm"].append(letra_norm)
    contents["similarity"].append(simililarity)

    if w < 10 or h < 10:  # contornos muito pequenos não serão mostrados
        if i < (10 * 10):
            ax[i].axis("off")
        contents["check_is_valid"].append(False)
        continue
    contents["check_is_valid"].append(True)

    if i < (10 * 10):
        ax[i].imshow(letra_norm, cmap='gray')
        ax[i].axis("off")
        ax[i].set_title(f"Sim: {simililarity:.2f}")
fig.savefig(path_assets / "atv02-q08-i02_k04.png", dpi=400, bbox_inches='tight')
fig.suptitle("Amostra de letras e similaridade calculada")
```

2.4) Conclusão

Aplicando o *threshold* de 0.5, não foi encontrado nenhuma letra **A** pelo algoritmo

```
ks = np.where((np.array(contents["similarity"]) >= 0.5) &
(np.array(contents["check_is_valid"]) == True))[0]
n = math.ceil(math.sqrt(len(ks)))

if n != 0:
    fig, axes = plt.subplots(
        n,
        n,
        figsize=(n * 3, n * 3)
    )


    for value, _axes in zip(ks, axes.flatten()):
        print("test")
        _axes.imshow(contents["letra_norm"][value], cmap="gray")
        _axes.set_title(f"Contorno: {value}")

    for _axes in axes.flatten():
        _axes.axis('off')

    text = f"Letra 'A' {'encontrada' if len(ks) >= 1 else 'inexistente'}"
    text += f" | Total: {len(ks)}"
    fig.suptitle(text, fontsize=16, weight='bold')
    fig.savefig(path_assets / "atv02-q08-i02_k05.png", dpi=400, bbox_inches='tight')
    plt.show()
else:
    fig, axes = plt.subplots(
        1,
        1,
        figsize=(1, 8)
    )
    axes.set_title("Letra 'A' não encontrada")
    axes.axis('off')
    fig.savefig(path_assets / "atv02-q08-i02_k05.png", dpi=400, bbox_inches='tight')
```

Letra 'A' não encontrada

Questão 09

Considere a imagem `cameraman_pattern.png`. Tente eliminar o padrão de linhas que aparece nela, da melhor forma possível, usando:

a) uma solução aplicada no domínio da frequência;

b) uma solução aplicada no domínio espacial.

R.:

- **Obs.:** o código em notebook para essa questão pode ser encontrado clicando no link.

Questão 10

Análise Escala-Espaço: Observamos o mundo ao nosso redor em diferentes escalas. Sempre temos, à nossa frente, objetos mais próximos e objetos mais distantes. Nesse sentido, tomando como base a teoria de Marr, Pietro Perona e Jitendra Malik propuseram um método de detecção de bordas, usando a teoria escala-espaço. A ideia principal é que, em diferentes escalas da imagem, diferentes bordas se sobressaem. Essa variação de escala poderia ser conseguida com a aplicação de sucessivos resizes na imagem. No entanto, como cada resize tem como consequência, além da mudança de dimensões de uma imagem, a perda de detalhes, essa operação foi substituída pela aplicação de filtros Gaussianos de diferentes parâmetros. À medida que o filtro se torna mais forte (maior o desvio padrão), mais embaçada a imagem fica o que gera a perda de detalhes esperada.

Comprove a aplicação da teoria escala-espaço na detecção de bordas com o seguinte experimento: na imagem cameraman.png, aplique três filtros gaussianos diferentes (parte de um mais fraco e vá até um mais forte), detecte as bordas de cada um com o detector de Canny (precisa usar os mesmos parâmetros para todas as imagens) e, por fim, avalie os resultados finais, verificando se diferentes bordas foram encontradas.

OBS: Não se preocupe em achar um resultado final de boa qualidade; não é esse o objetivo do experimento

R.:

- **Obs.:** o código em notebook para essa questão pode ser encontrado clicando no link.

O intuito desse experimento é verificar a detecção das bordas apos a aplicação de três filtros gaussianos, e analisar o contorno encontrado apos cada filtro. Dessa forma, o esperado é que o nível de detalhe caia, a medida que se aumenta o sigma do filtro gaussiano, causando um efeito de perca de detalhes.

- **Leitura**

```
img_names = ["cameraman.png"]
imgs = {
    i.split(".")[0]: cv2.imread(path_imgs_atv / i, cv2.IMREAD_GRAYSCALE) for i in
img_names
}
```

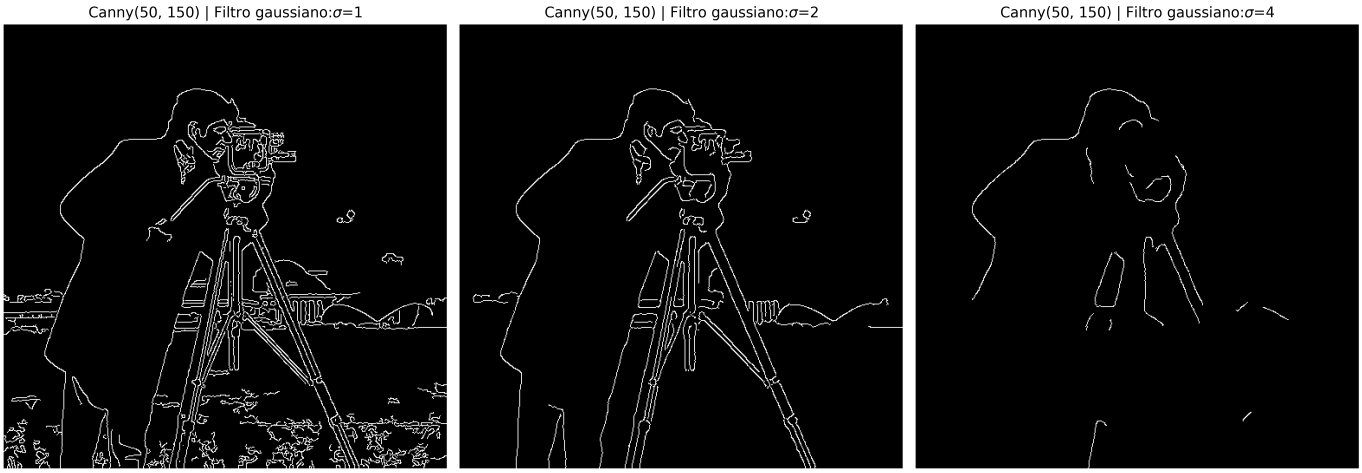
- **Configurando parâmetros**

```
# Sigmas para o filtro gaussiano
sigmas = [1, 2, 4]
```

```
# Parametros para o método de deteccao de canny
low = 50
high = 150
```

- **Implementação e geração de gráficos**

```
fig, ax = plt.subplots(1, len(sigmas), figsize=(16, 8))
ax = ax.flatten()
for i, sigma in enumerate(sigmas):
    # Aplica filtro Gaussiano com sigma definido
    blurred = cv2.GaussianBlur(
        imgs["cameraman"],
        (0, 0),
        sigmaX=sigma,
        sigmaY=sigma
    )

    # Detecta bordas com Canny
    edges = cv2.Canny(
        blurred,
        low,
        high
    )

    # Exibe os resultados
    ax[i].imshow(edges, cmap='gray')
    ax[i].set_title(f'Canny{low, high} | Filtro gaussiano:' + rf'$\sigma$={sigma}')
    ax[i].axis('off')
fig.tight_layout()
fig.savefig(path_assets / "atv02-q10-01.png", dpi=400, bbox_inches='tight')
plt.show()
```



- **σ = 1**: existe diversas bordas, mostrando um nivel de detalhe bem significante, preservando pontos princiapis da imagem e secundários também. Isso se da possivelmente pelo valor de σ baixo, causando um efeito de emabaçamento menor.

- **σ = 2**: destaque nos detalhes principais, sumindo parcialmente traços mais secundários como os da grama. O filtro Gaussiano mais forte suaviza regiões pequenas, e o detector de bordas começa a destacar contornos mais marcantes.

- **σ = 4**: apenas as bordas principais da imagem são visíveis. Com o aumento da suavização, diversos detalhes desaparecem, restando apenas as transições de contraste mais evidentes.

Essa progressão demonstra que, ao aumentar o valor de σ no filtro Gaussiano, esta focando na imagem apenas os pontos mais caracteristicos, ou seja, apenas bordas maiores e mais importantes são conservadas. Dessa forma, diferentes bordas aparecem em diferentes escalas, demonstrando a ideia da análise escala-espaço como uma ferramenta bem interessante na segmentação de imagens com muitos objetos visuais.